

# Smoothing the Jagged Boundaries in FastSAM Predictions

Debanjan Saha    Apurva Rai

debanjansaha@iisc.ac.in    apurvarai@iisc.ac.in  
Sr.: 23607    Sr.: 26381

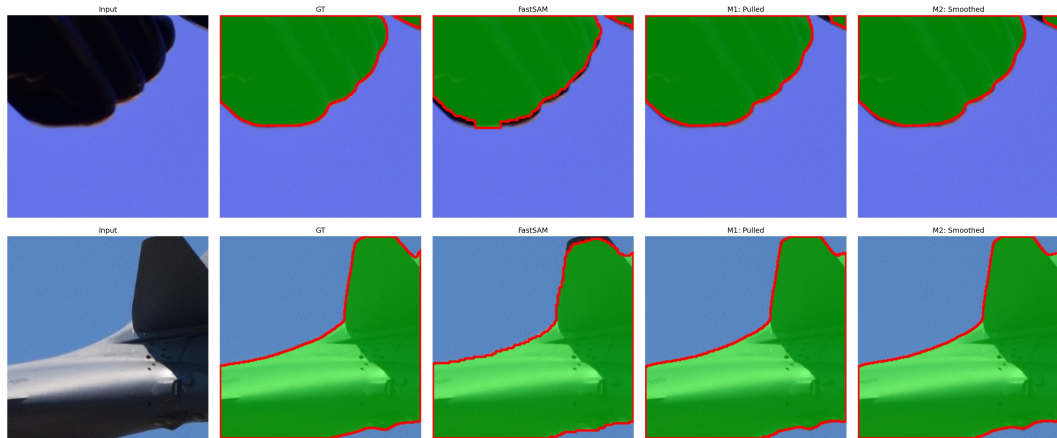
July 13, 2026

# Introduction and Motivation

- ▶ **The Problem:** Fast Segment Anything Model (FastSAM) provides rapid object segmentations, but its predicted masks frequently suffer from poor boundary quality.
- ▶ **Key Inaccuracies:** Boundaries exhibit jagged, staircase artifacts and often fail to snap tightly to the true physical edges of objects.
- ▶ **The Objective:** To resolve these boundary inaccuracies natively while preserving FastSAM's speed advantage, making the masks suitable for high-precision downstream tasks.
- ▶ **Our Solution:** A lightweight ( $\sim 200$  kb), two-stage local refinement neural network. By processing only localized patches around the initial boundary, we correct loose snapping and iron out jagged edges with minimal computational overhead.  
**Input:** Image (patch) and FastSAM mask, **Output:** Refined mask with pulled and smooth boundary.

**Note:** All experiments conducted locally (i7-12650H, RTX 3050Ti) or via Kaggle (2x T4 GPU).

# Visualizing the Goal: Loose Boundaries to Perfect Snaps



**Core Objective:** Transform FastSAM's rough, loosely snapped, and jagged initializations (middle) into precise contours that perfectly adhere to the actual photometric edges of the object (rightmost).

# Related Work & Baselines

## Classical Digital Image Processing (DIP)

- ▶ Goal: Smooth jagged artifacts AND pull loose boundaries to true edges.
- ▶ **Guided Filtering:** Smoothed well, but failed to shift boundary (under-segmentation). Blur caused leakage.
- ▶ **GrabCut:** best in classical, but slow ( $\sim 3s$  per object). Unsuitable for real-time.
- ▶ **Watershed:** Extremely fast ( $\sim 0.04s$ ), but highly susceptible to noise and shadows (severe boundary leaks).
- ▶ **Others (Mean-Shift, FMM, Active Contours):** Distorted shapes or stuck in local minima.

## Recent Literature (FastSmoothSAM)

- ▶ Focuses only on smoothing already-accurate boundaries.
- ▶ Lacks pulling mechanism for heavily deviated FastSAM masks.

# Baseline Visuals & Deep Learning Attempt

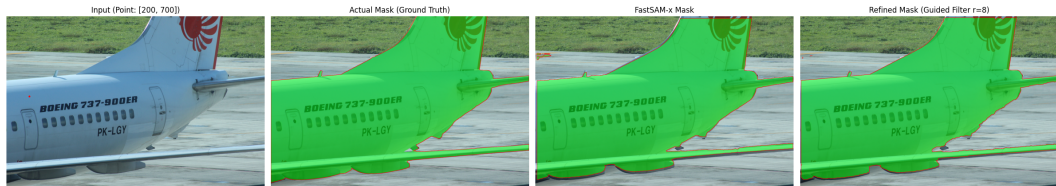


Figure: GrabCut Optimization: Better but computationally expensive ( $\sim 3s$ ).

## Single-Pass CNN Approach

- ▶ Tested standard feed-forward CNN (single forward pass).
- ▶ **Result:** Failed to pull boundaries effectively.
- ▶ **Reason:** Single pass lacks spatial reach/dynamic feedback to drag boundaries over large pixel distances.
- ▶ **Conclusion:** Iterative, recurrent approach is mandatory.

# Model Assumptions & Initial Training Strategy

## Fundamental Architecture Assumptions

- ▶ Ground truth is on/close to the actual photometric edge (Failure case: SuperGT).
- ▶ FastSAM predicted boundary is close to *some* edge (Failure case: out of visual range of our model).
- ▶ FastSAM predicted boundary is closer to the *actual edge* than some other edge (Failure case: parallel edges).

## Initial Full-Image Attempt

- ▶ Trained on complete, full-resolution image/mask pairs.
- ▶ **Failure:** Model failed to learn; outputs remained identical to initial FastSAM inputs.
- ▶ **Cause:** Capacity consumed by global structural context, causing the network to ignore localized high-frequency artifacts (Global Context Entanglement).

# The Patch-Based Strategy & Strict Filtering

## Why Patches?

- Forces exclusive focus on local boundary correction, resolving global context entanglement.

## Dataset Generation & Constraints (256 × 256 Crops)

- **Image-Level Filter:** Baseline FastSAM IoU  $\geq 0.85$  and Area within  $\pm 12\%$ .
- **Boundary-Centric:** Crop centers placed exactly on the true GT edge.
- **Area Balance:** GT mask strictly occupies 35%–65% of the patch area.
- **Ultra-Strict Patch IoU:** Local patch FastSAM IoU  $\geq 0.95$  (Area diff  $\pm 5\%$ ).
- **Anti-Padding:** Shift window inward at image borders to avoid black padding (prevents learning artificial straight edges).
- **Spatial Diversity:** Patch centers kept  $\geq 100\text{px}$  apart to prevent overfitting.

# Methodology: Stage 1 - The Boundary Puller

## Objective & Intuition

- ▶ Actively pull the loosely snapped FastSAM mask toward photometric edges.
- ▶ Deep, narrow recurrent CNN emulating classical active contours (snakes), but driven by learned semantic features.
- ▶ Uses Dilated Residual blocks to expand the receptive field and *see distant edges*.

## Recurrent Update Rule

- ▶ Iteratively updates the mask logits  $M$  over  $T$  timesteps using image  $I$ :

$$M_{t+1} = M_t + \mathcal{F}_{\text{loop}}(\text{Concat}(\Phi_{\text{img}}(I), \Phi_{\text{mask}}(\sigma(M_t))))$$

## Localized Soft-Boundary Loss

- ▶ Standard BCE is dominated by interior pixels.
- ▶ **Solution: Spatially Weighted BCE.** We apply morphological dilation/erosion and blurring to create a boundary weight map  $W$ , penalizing only edge misalignments.

$$L_{\text{puller}} = \frac{1}{N} \sum \left( \text{BCE}(\hat{Y}_{\text{puller}}, Y) \odot W \right)$$



# Methodology: Stage 2 - The Smoother & Discarded Approaches

## Stage 2: The Boundary Smoother

- ▶ **Intuition:** Puller's sensitivity to high-frequency gradients causes jaggies. The Smoother acts as a learned morphological filter to iron these out.
- ▶ **Architecture:** Shallow CNN taking the RGB patch and the thresholded Puller output.
- ▶ **Loss:** Combines soft-boundary BCE with a Local Density Scatter penalty to eliminate isolated artifacts and holes:

$$L_{\text{smoother}} = \text{BCE}(\hat{Y}_{\text{smooth}}, Y) + \lambda \cdot L_{\text{density}}$$

## Discarded Approach: Wavelet Transform

- ▶ Attempted to inject high-frequency features via Wavelet Transform (inspired by Random Fourier Features in PINNs) to capture fine corners.
- ▶ **Result:** Captured corners well, but caused catastrophic wobbling along straight edges due to hypersensitivity to pixel noise. Discarded.

# Acceleration & Optimization

## The Bottleneck Analysis

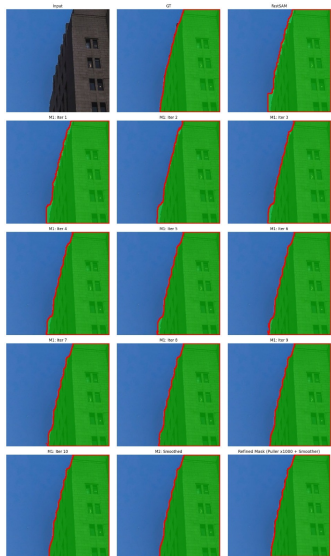
- ▶ **GPU (PCIe Bottleneck):** Batched inference (0.25s) was barely faster than sequential (0.26s). Data transfer overhead  $\gg$  actual compute time of tiny model.
- ▶ **CPU PyTorch (Cache Thrashing):** Aggressive multi-threading/large batching degraded performance ( $> 3.5s$ ) because data volume exceeded the 24MB L3 Cache, forcing slow System RAM transfers.

## Optimization Strategy: ONNX Fusion

- ▶ Shifted to CPU deployment for edge compatibility by unrolling the 5 recurrent iterations and fusing both models into a single ONNX graph. This eliminated Python execution overhead and kept intermediate tensors locked in high-speed L1/L2 cache.

| Method                     | Hardware   | Time per Image | Status                     |
|----------------------------|------------|----------------|----------------------------|
| PyTorch Unoptimized        | CPU        | $\sim 1.81$ s  | Slower than FastSAM        |
| Separate ONNX Models       | CPU        | $\sim 1.08$ s  | Memory overhead            |
| <b>Fused ONNX Pipeline</b> | <b>CPU</b> | $\sim 0.85$ s  | $> 2\times$ <b>Speedup</b> |

# Qualitative Results: The Pulling Process



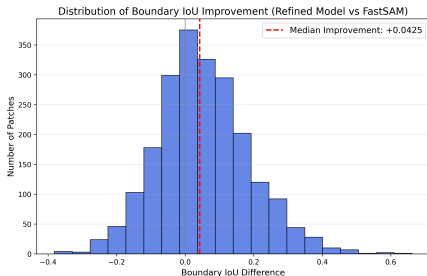
## Visualizing the Puller

- ▶ Boundary incrementally snaps to the true photometric edge.
- ▶ **Robustness:** Once locked onto the physical edge, the contour stabilizes. Notice the identical boundaries at the 10<sup>th</sup> and 1000<sup>th</sup> iterations.

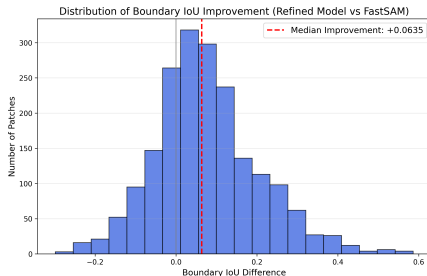
## Evaluation Methodology & Outlier Rejection

- ▶ Evaluated strictly on localized  $256 \times 256$  patches.
- ▶ Discarded catastrophic FastSAM initializations (e.g., completely wrong object).
- ▶ **Outlier Rejection:** Sorted by final Smoother Boundary IoU and rejected the bottom 10% as unrefinable outliers (analyzed in Limitations).

# Quantitative Results & Ablation



SA-1B Dataset (Median: +0.0425)

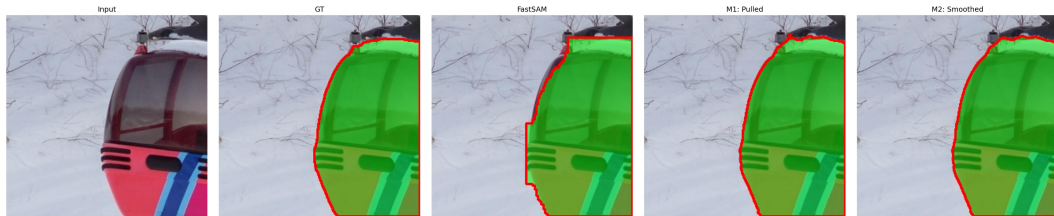


DIS5K Dataset (Median: +0.0635)

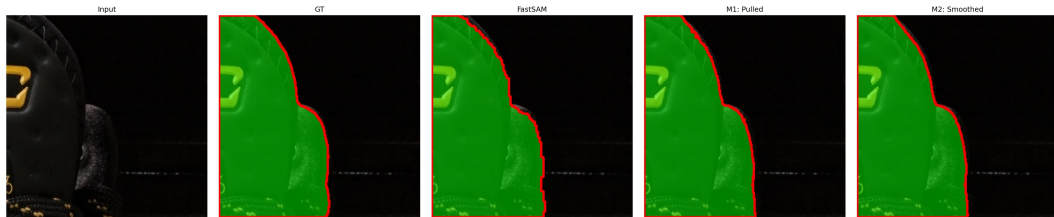
- Steady B-IoU increase confirms recurrent architecture effectively snaps masks to edges.

| Metric            | SA-1B (Baseline) | SA-1B (Refined) | DIS5K (Baseline) | DIS5K (Refined) |
|-------------------|------------------|-----------------|------------------|-----------------|
| Global IoU        | 0.9693           | 0.9751          | 0.9443           | 0.9568          |
| Boundary IoU      | 0.6651           | <b>0.7176</b>   | 0.6146           | <b>0.6906</b>   |
| Median B-IoU Imp. | +0.0425          |                 | +0.0635          |                 |
| Peak B-IoU Imp.   | <b>+0.6622</b>   |                 | <b>+0.5862</b>   |                 |

# Qualitative Results: Pulling & Smoothing

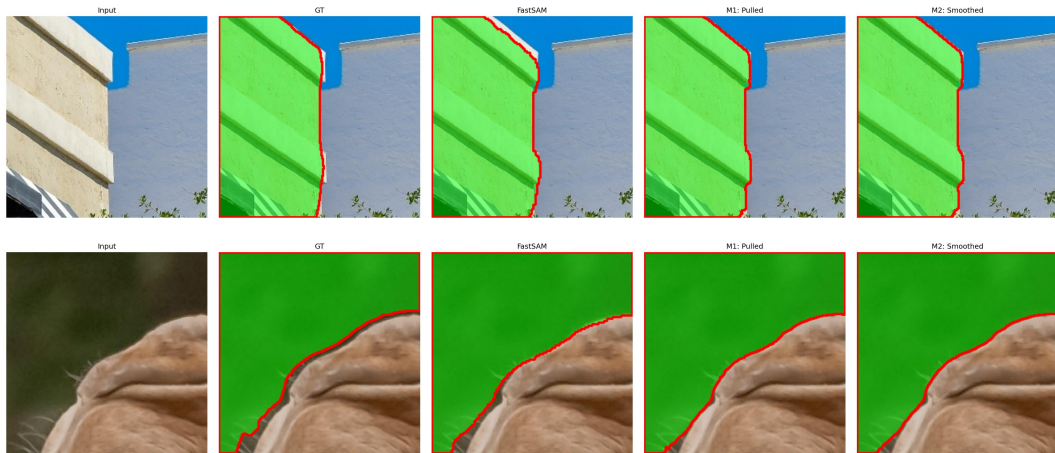


**Effective Pulling:** Snapping heavily deviated FastSAM contours to true object edges.



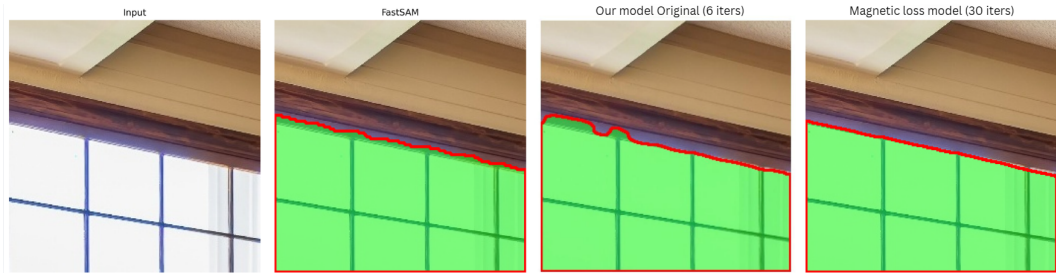
**Effective Smoothing:** Ironing out high-frequency jagged artifacts on otherwise well-localized masks.

# Qualitative Results: Superior to Ground Truth (SuperGT)



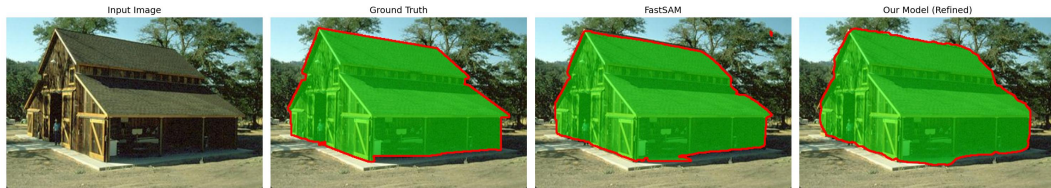
**SuperGT Examples:** Our two-stage refinement correctly identifies and snaps to actual object contours that the dataset's original Ground Truth either missed or annotated poorly.

# Limitation Mitigation: Parallel Edges

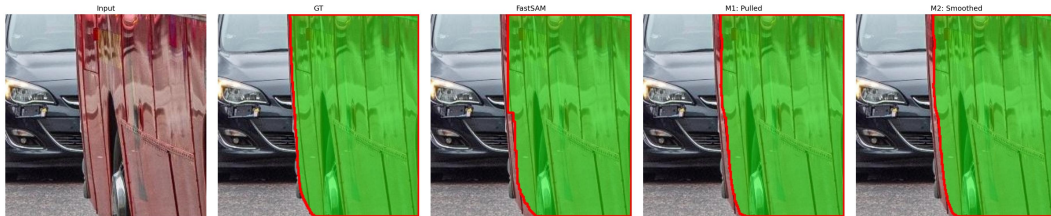


- **Issue:** Greedy local snapping causes the predicted contour to jump back and forth between close parallel edges.
- **Mitigation (Magnetic-Edge Loss):** Introduced a global penalty based on Sobel image gradients. It heavily penalizes predicted boundaries that settle on flat, featureless regions between the actual edges.
- **Trade-off:** Successfully resolves the alternating boundary, but requires significantly more iterations to converge ( $\sim 30$  iters).

# Limitations: Resolution & Receptive Field



**HQSeg Dataset Failure:** Fails on low-resolution, highly compressed images. The model was trained strictly on high-res SA-1B patches and relies on sharp, multi-pixel gradients.



**Receptive Field Limit:** The initial FastSAM boundary is too far from the true edge, falling completely outside the Puller's local visual range of dilations.



# Conclusion

## Summary of Contributions

- ▶ Developed a lightweight, two-stage local refinement architecture (Boundary Puller + Smoother) that actively snaps FastSAM's jagged contours to true photometric edges.
- ▶ Achieved a  $2.1\times$  speedup through ONNX fusion compared to PyTorch CPU, refining boundaries in  $< 50\%$  of FastSAM's base inference time, making it viable for edge CPU deployment.

## Team Contribution

- ▶ The project workload, including pipeline development, model training, and optimization, was shared equally ( $\sim 50/50$ ) between Debanjan Saha and Apurva Rai.

**Thank You!**

Codebase & Interactive Demo available at:

<https://github.com/deban9017/FastSAM-Smooth-boundary>